

dpr: Automatic Parallel Execution of Ordinary Python Programs on a Local Network of Computers

Project Report – dpr version 2.1.0 – September 2026

Abstract—Most Python programs use a single processor core, even on computers with many cores and in places full of idle machines. Tools that spread work across several computers exist, but they usually require the program to be rewritten for a new programming model. This report describes **dpr**, a system that takes an ordinary Python file, finds the parts that do not depend on each other without running it, and runs those parts at the same time on other computers in the same local network. **dpr** puts correctness first: a part is only moved to another computer when the analysis proves, or the programmer declares with a one-line marker, that it is self-contained; everything else runs in its original order. As long as these declarations are true and the program does not depend on the computer it runs on, the result is the same as in a normal run. The system consists of about 23 000 lines of Python in ten components, including a static program analyzer, a scheduler, a coordinator, worker processes, an encrypted peer-to-peer data channel and a two-command user interface. It is protected by mutually authenticated TLS 1.3, an HMAC challenge at login, pinned join codes and explicit approval of every new machine, and it is verified by 1841 automated tests on Python 3.12 and 3.13 and by end-to-end stress tests. On two laptops with four logical cores each, **dpr** ran a 214-task mixed workload 3.1 times faster than plain Python and compute-heavy workloads 2.8 to 3.0 times faster, while workloads with no parallel work, with very small tasks, or dominated by moving data became slower.

Index Terms—Distributed computing, parallel execution, Python, static analysis, task graphs, scheduling, transport layer security, local area networks.

I. INTRODUCTION

Computers are faster than ever, but a normal Python program cannot use most of that speed. The standard Python interpreter, CPython, runs Python code in one thread at a time because of its *global interpreter lock* (GIL) [1]. A program that is started with `python program.py` therefore keeps one processor core busy, even on a machine with sixteen. Other computers on the same desk or in the same room are often idle at the same time.

Python’s own `multiprocessing` module [2] offers one way around this by running code in several processes of one computer. Other systems, such as MapReduce [3], Spark [4], Dask [5] and Ray [6], spread work over several computers. All of them ask the programmer to restructure the program: to create pools, submit futures, describe data sets or decorate every remote

call; the distributed ones also need a cluster to be set up and maintained. For a student, a researcher or a small team with a few ordinary computers, that effort is often larger than the program itself.

dpr (a *distributed Python runtime*) takes a different approach. The user gives it an ordinary Python file. **dpr** reads the file without running it, builds a map of which statements depend on which, and runs the statements that are independent of each other at the same time on other computers of the same local network. Functions that **dpr** cannot check by itself can be marked with a one-line marker, `@task`, which does nothing when the file is run normally. As long as these markers are true and the program does not depend on the computer it runs on (for example, on files outside its folder), the result is the same as running the file normally; programs with enough independent work simply finish sooner.

This report describes the whole project in simple terms. Its main contributions are:

- A *conservative program analyzer* that turns one Python module into a task graph and only allows a task to run elsewhere when it is proven or declared to be self-contained (Section V).
- A complete *runtime*: planner, scheduler, coordinator, workers and a peer-to-peer data channel, all with bounded resources (Sections VI–IX).
- *Secure cluster formation* for a local network, with pinned join codes, explicit approval and permanent removal of machines (Sections X and XII).
- A *minimal user interface* of two outside commands and seven session commands, with automatic management of processes, ports, credentials and storage (Sections XI and XIII).
- An extensive *verification* effort of 1841 tests plus stress tests (Section XV), and a *benchmark study* on real hardware (Section XVII).

II. BACKGROUND

A. Processes, Threads and the GIL

A *process* is a running program with its own memory. A *thread* is a path of execution inside a process; the threads of one process share its memory. In CPython, the GIL lets only one thread execute Python code at any moment, so threads help with waiting (for example, for the network) but not with computing. Separate processes, on the other hand, each have their own interpreter and can compute in parallel [2]. **dpr** therefore runs work in separate processes, and these processes can live on different computers.

TABLE I
REQUIREMENTS AND HOW THEY ARE MET

Requirement	How dpr meets it
Correct results	Only proven or declared self-contained tasks run in parallel; everything else keeps its original order.
No restructuring	Statement-level analysis of the unchanged file; an optional one-line <code>@task</code> marker.
Simple to use	Two commands outside a session, seven inside it.
Secure by default	TLS 1.3 with mutual certificates, HMAC challenges, pinned join codes, host approval.
Self-managing	Processes, ports, credentials, caches and logs are handled automatically.
Bounded resources	Queues, messages, files and values all have limits; bursts of messages slow senders down instead of failing.
Portable	Windows, Linux and macOS; only Python 3.12+ (and <code>openssl</code> on the host).

B. Task Graphs

A program can be described as a set of *tasks* (pieces of work) and *dependencies* (task B needs the result of task A). Drawn as boxes and arrows, this forms a *directed acyclic graph* (DAG): arrows only point forward, so there is always an order in which every task comes after the tasks it needs, called a topological order [7]. Tasks without a path between them are independent and may run at the same time.

C. Measuring Parallel Speed

If a program takes T_{plain} seconds normally and T_{dpr} seconds with dpr, the *speedup* is

$$S = T_{\text{plain}}/T_{\text{dpr}}, \quad (1)$$

and with p processor cores working, the *parallel efficiency* is $E = S/p$. Amdahl's law [8] explains why S stays below p : if only a fraction f of the work can run in parallel, then

$$S \leq \frac{1}{(1-f) + f/p}. \quad (2)$$

In a distributed system, the time to start tasks and to move data between machines lowers the speedup further. Both effects are visible in Section XVII.

D. Related Systems

MapReduce [3] and Spark [4] process large data sets on big clusters through special operators. Dask [5] builds task graphs from collections and delayed functions, and Ray [6] runs remote functions and actors declared with its API. Python's own `multiprocessing` and `concurrent.futures` modules run functions in other processes on one machine. All of them expect the program to be written for them. dpr instead analyzes an unchanged program statement by statement, needs no third-party packages, and is aimed at a small, trusted group of ordinary computers on one local network.

III. REQUIREMENTS AND DESIGN PRINCIPLES

Table I lists the requirements that guided every design decision and how the final system meets them.

A single principle connects them: *fail closed*. Whenever dpr cannot be sure that something is safe, it takes the safe path: it

TABLE II
COMPONENTS OF THE CODE BASE

Package	Purpose	Lines	Tests
<code>dag_runtime</code>	Reads a program and builds its task graph	2 776	337
<code>execution</code>	Turns the graph into an execution plan	654	141
<code>scheduler</code>	Chooses a worker for each ready task	761	183
<code>coordinator</code>	Authoritative state of runs, tasks, transfers	5 299	295
<code>worker</code>	Runs tasks, stores and sends values	4 923	49
<code>networking</code>	Control connections and services	2 078	80
<code>protocol</code>	Typed messages, encoding, limits	2 389	638
<code>program_package</code>	Packs, ships and caches programs	798	18
<code>runtime_security</code>	TLS policies and HMAC authentication	280	11
<code>dpr</code>	Sessions, commands, joining, credentials	2 839	89
Total	57 source files (+3 build tools)	22 797	1 841

Lines: production code, without tests, examples and benchmark scripts; the three build tools add 120 lines. Tests: automated tests located in the package (some test other packages too).

runs code in its original order, rejects an unexpected message, or stops a run with a clear explanation. It never guesses.

IV. SYSTEM OVERVIEW

A. Roles

Every computer runs a *session*: the `dpr>` prompt that the user types into. One computer is the *host*. Its session starts a background *coordinator node*, which keeps the authoritative state of the cluster, and an *enrollment service*, which lets new machines ask to join. The other computers are *workers*; their sessions start a *worker node*, which executes tasks, stores their results and exchanges data with other workers. Fig. 1 shows the parts and how they connect.

B. Components

The code is divided into ten components (Python packages). Each runtime component has its own technical documentation of what it does and promises; five of them have a formal contract document. Table II gives their purpose and size; Fig. 2 compares production code and test code.

C. The Life of a Run

Fig. 3 follows one program from the moment the user types `run` to the printed result. The session on the host packs the program's folder, analyzes the program and submits the package to the coordinator, which analyzes the program again and checks that it arrives at the same plan. The coordinator ships the program to the workers, the scheduler places ready tasks, the workers execute them and exchange results directly, and the session finally prints the output and saves a result file. Values that are no longer needed are then released on the workers, and the run is archived.

V. PROGRAM ANALYSIS

The analyzer (`dag_runtime`) is the heart of dpr: it decides what may run in parallel. It works entirely on the program text and never executes it.

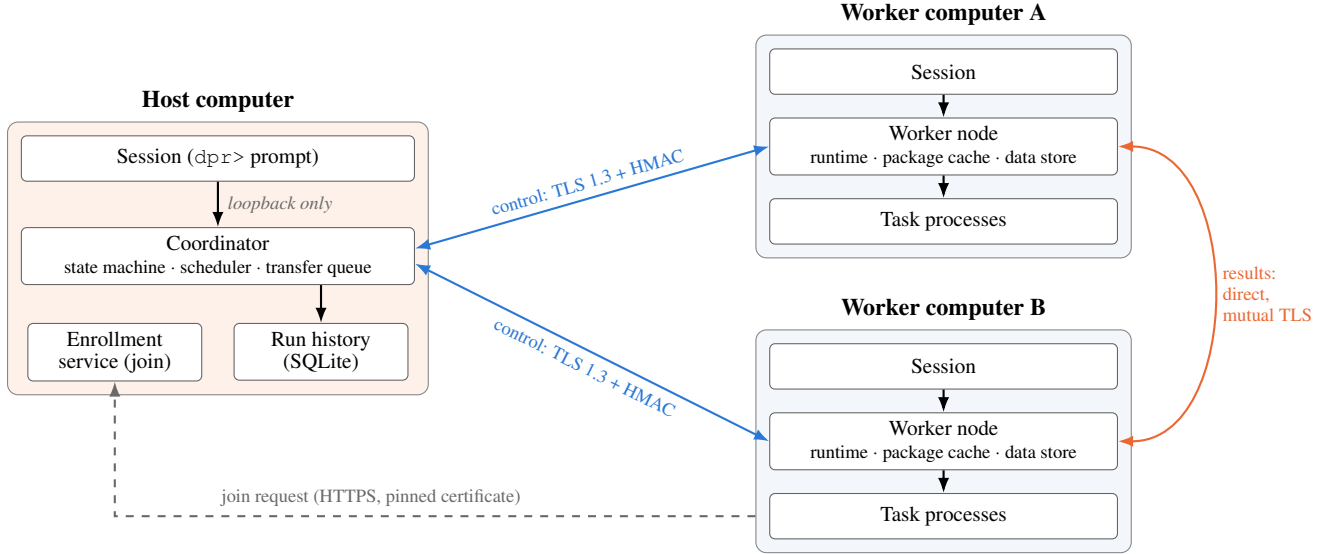


Fig. 1. Architecture of dpr. Every computer runs a session. The host’s session starts the coordinator and the enrollment service; each worker’s session starts a worker node. Control messages travel between the coordinator and each worker over TLS 1.3 with an extra HMAC check; results travel directly between workers over their own mutually authenticated TLS connections. Programs can only be started from the host itself (loopback).

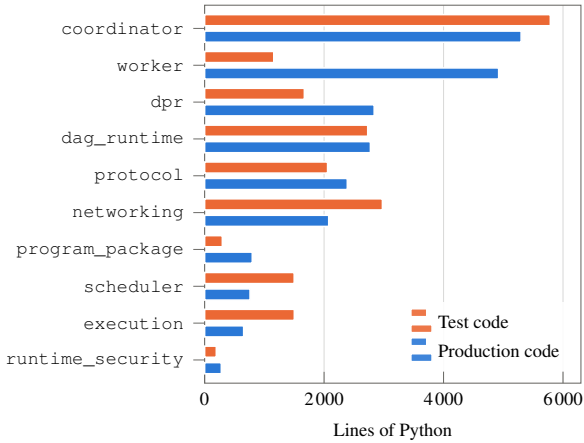


Fig. 2. Size of each component: production code and the test code kept beside it. Almost as much code tests the system (19 878 lines) as implements it (22 797 lines).

A. From Statements to Tasks

The analyzer parses the file with Python’s own `ast` module [9] and walks through the top-level statements in order. Every top-level statement becomes one *task*, except plain function definitions, which are recorded but not turned into work. A statement that contains several calls, such as `r = combine(f(1), f(2))`, stays one task, so Python’s evaluation order and short-circuit rules (for example in `a and b()`) are kept exactly.

B. Dependencies

Every assignment creates a new *version* of a variable, like `total#1`, `total#2`. A task that reads a variable depends on the task that produced the version it reads. The graph has three kinds of edges:

- **Data:** the task reads a value made by an earlier task.

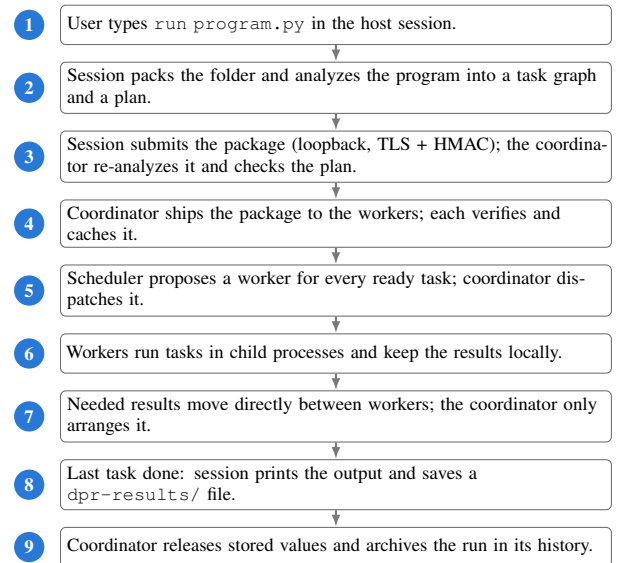


Fig. 3. The life of a run, from typing `run` to the saved result. Steps 5 to 7 repeat until every task of the program has finished.

- **State:** the task needs the program’s shared state (for example, after a list was changed in place).
- **Order:** an effect that the analyzer cannot see through must keep its original position.

The graph is checked to be acyclic and complete, and it can explain why two tasks are or are not allowed to run at the same time.

C. Proving a Task Self-Contained

A task may only run in another process, on another computer, if it is *self-contained*: everything it needs comes in as values, everything it produces goes out as values, and it changes nothing

Listing 1. A program that dpr runs in parallel. It also runs unchanged with plain python on a computer where dpr is installed.

```

from dag_runtime import task

@task
def count_primes(start, end):
    count = 0
    for n in range(max(start, 2), end):
        if all([n % d for d in
                range(2, int(n ** 0.5) + 1)]):
            count += 1
    return count

a = count_primes(0, 250_000)
b = count_primes(250_000, 500_000)
c = count_primes(500_000, 750_000)
d = count_primes(750_000, 1_000_000)

total = a + b + c + d
print("primes:", total)

```

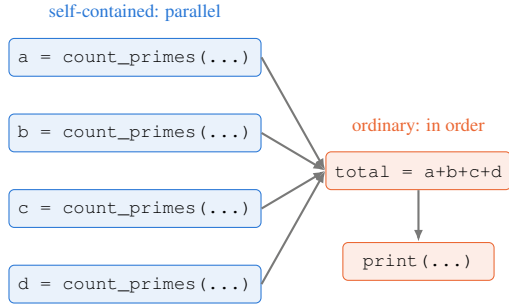


Fig. 4. Task graph of Listing 1. The four calls are independent and self-contained, so they run at the same time on four slots. The sum and the print are ordinary code and run afterwards, in order.

that other tasks use. The analyzer proves this for a bounded but useful family of code: integer and floating-point arithmetic with known types, comparisons, range, len, sum, abs, min and max, list comprehensions over known data, lists that a function builds itself with append, and local if, for and while blocks inside functions. Division is only accepted when a zero divisor is ruled out. Anything outside this family is treated as unknown.

D. The @task Contract

For code the analyzer cannot check itself, such as text processing, file reading or library calls, the programmer can mark a function with @task from dag_runtime. The marker does nothing at run time; it is a promise that the function is self-contained. Only the marker imported from dag_runtime is trusted; a different decorator that happens to be called task is not. Listing 1 shows a complete program and Fig. 4 its task graph.

E. Where the Analysis Stops

The analyzer never guesses, and some statements hide what they do. An import can run arbitrary code while the module loads [10]; print, file access and calls to unknown functions have effects outside the program; and a top-level if, for, while, try, with, class definition or augmented assignment (x += 1) may change any variable. After such a statement,

TABLE III
TASKS FOUND IN THE BENCHMARK PROGRAMS

Workload	Tasks	Self-contained (parallel)
01 Latency	3	2
02 CPU, coarse-grained	35	34
03 CPU, fine-grained	259	258
04 Data movement	11	10
05 Map-reduce	51	50
06 Dependency chain	14	13
07 Mixed stress test	214	212

the analyzer no longer trusts what it knew about the program's variables, so the statements that follow run one after another. The analysis document even gives a concrete reason: code run by an import could install an object whose clean-up routine later replaces a variable, so a fresh-looking assignment proves nothing.

A few features make the whole program run as a single piece from the start, wherever they appear in the file, even inside a function: generator expressions, async and await, yield, and imports of the threading, asyncio, multiprocessing or concurrent modules. Dynamic code (eval, exec, globals(), locals(), from x import *) keeps the rest of the program, from that line on, in one piece. In all these cases the program still runs correctly, only not in parallel. Programs that parallelize well therefore share one shape: function definitions (with their imports inside them), then the independent heavy calls, then the lines that combine and print the results.

F. Analysis of the Benchmark Programs

Table III shows how the analyzer split the seven benchmark programs of Section XVII. Almost every task was found to be self-contained, because the programs follow the shape described above.

VI. EXECUTION PLANNING AND SCHEDULING

A. Execution Plan

The execution component turns the task graph into an *execution plan*: one manifest per task, listing its inputs and outputs, the code it needs and its *execution mode*. The plan is identified by a hash of the exact source, the file name and the package, so a worker can never run a task against a different version of the program. There are three modes:

- *Isolated candidate*: a self-contained task. It runs in a fresh process on any worker, and it can be retried safely.
- *Shared context*: ordinary code that needs the program's shared variables. It runs inside one long-lived Python process on one worker, in original order.
- *Native region*: a block the analyzer treats as opaque (a top-level loop, for example). It also runs inside the shared process, as one step.

B. Scheduler

The scheduler is a pure function: given a snapshot of the cluster, it proposes a worker for each ready task and changes nothing.

The coordinator then re-checks and reserves each proposal before sending it. First, hard rules remove impossible choices: the worker must be online and accepting work, have a free slot (one slot per logical processor core), have the program prepared, support the task’s mode, and be able to obtain all inputs. Then tasks are ordered: tasks that have waited eight scheduling rounds come first (fairness), then tasks with fewer possible workers, then tasks with longer chains of work after them. Workers are ranked for each task in this order:

1. all inputs already on that worker;
2. fewer inputs of unknown size to fetch;
3. fewer bytes to fetch, then fewer inputs to fetch;
4. more free slots;
5. lower memory load, then lower processor load (in 10 % bands);
6. more free memory, then more cores; finally the worker name.

Data locality therefore always beats load: a task goes where its data already is whenever that worker has a free slot.

VII. THE COORDINATOR

A. Authoritative State

The coordinator is the single source of truth. It records every run, task, *attempt* (one try at running a task), transfer and stored value, and every worker connection with a *generation* number that grows each time the worker reconnects. Every incoming event is validated and classified as *applied*, *duplicate* or *stale*. A message from an older generation of a worker is stale and changes nothing, so a slow or reconnecting machine can never corrupt a run.

B. All-or-Nothing Changes

Each operation first *stages* everything that could fail, such as creating identifiers, building messages and checking queue space, and only then changes the state. If any step fails, the state is exactly as before, and the operation can simply be tried again.

C. Time Limits and Retries

Workers send a heartbeat every second; a worker silent for 30 s is treated as lost. Other deadlines guard each step: 60 s to prepare a program, 30 s to acknowledge a dispatched task, 30 s to start it, 30 s to confirm a cancellation, and 60 s per transfer step plus one second for every 256 KiB of the value. A task gets at most two attempts. It is retried when its input was unavailable, when the worker’s environment did not match, or when an execution error or a lost worker interrupted it, never when the program itself raised an exception (that would fail again), and never when it was shared or native code that had already started (it may not be safe to repeat).

D. Transfer Admission Control

When a task needs a value that lives on another worker, the coordinator arranges a transfer. A task that gathers many results could create dozens of transfers at once. The coordinator therefore lets each worker take part in at most four transfers at a time in each direction, which matches the four data streams a worker runs. Further transfers wait in a queue on the coordinator, where

Listing 2. Releasing queued transfers (simplified from `coordinator.py`).

```
limit = self.operation_limits.max_transfers_per_worker
for key, prepare in tuple(self._queued_transfers.items()):
    record = self._transfers.get(key)
    if record is None or record.status.terminal:
        del self._queued_transfers[key] # ended in queue
        continue
    src = record.identity.source_worker_id
    dst = record.identity.destination_worker_id
    if sending.get(src, 0) >= limit or \
       receiving.get(dst, 0) >= limit:
        continue # no room on one side yet
    del self._queued_transfers[key]
    self._issued_transfers.add(key)
    record.phase_since = current # deadline starts now
    self._enqueue_message(destination, prepare)
    sending[src] = sending.get(src, 0) + 1
    receiving[dst] = receiving.get(dst, 0) + 1
```

they use no worker resources and no deadline runs. After every event that can finish a transfer, the queue is released in order (Listing 2).

E. Clean-up and History

When a run ends, the coordinator asks every worker to release the values the run left behind and waits for confirmation. Cancelled or failed runs also cancel their unfinished transfers on the workers at once. Finished runs are archived in an SQLite database [11]; the latest 1 000 runs are kept, and at most 2 048 runs are held in memory.

VIII. WORKERS

A. Running a Task

Each self-contained task runs in its own new Python process, started as `python -I -B isolated_child.py` inside a copy of the program’s folder. The `-I` flag starts Python in isolated mode, so environment variables and the user’s own package folder cannot change what gets imported; `-B` prevents writing cached bytecode. The child receives a tiny environment: a default program search path, the locale and time-zone settings and, on Windows, the system folder; nothing of the user’s, such as the home folder or credentials, is passed on. It reads its inputs from files, runs the task, and writes each output value to a file using Python’s `pickle` format [12]. The first 64 KiB of each output stream is captured, and the first 8 KiB of it is reported back to the host. Shared and native code runs in long-lived *context* processes instead, up to 128 per worker.

B. Limits Sized to the Machine

Each task has limits that stop a runaway task, sized so that a real workload is not cut short. By default a task may run for one day, use at least 2 GiB of memory (or the machine’s physical memory, if larger), use one day of processor time per core, and write files up to at least 1 GiB (or half of the free disk). The worker enforces the one-day limit itself. On Linux and macOS the other limits are applied through the operating system’s resource limits; on Windows, a job object applies the processor-time limit, and memory and file size are not limited. When a worker runs as the administrator (root) on Linux or macOS, tasks are started as the unprivileged user `nobody`.

C. Package Cache

Programs arrive as a deterministic ZIP archive with a manifest of SHA-256 digests [13]. Each worker verifies every file before use and keeps the unpacked program in a cache of 256 MiB that evicts old entries. A program folder may hold at most 16 MiB, 8 MiB per file and 2 048 files, with short, Windows-safe paths. Credential stores such as `.ssh` or `.aws`, the `.git` folder, caches, virtual environments and earlier results are never packed.

D. Data Store

Task results are stored as files, not in memory. A worker may hold a quarter of its free disk space for results, at least 256 MiB and at most 64 GiB, with at most 256 MiB per value. Every value is checked with SHA-256 when it is stored. The store is emptied when the worker restarts.

E. Data Plane

Workers send results to each other directly over a separate listener that uses TLS 1.3 with certificates on both sides. The coordinator issues an authorization for each transfer, and the receiver checks that the sender's certificate belongs to the expected worker before reading anything. The sender first sends a header with the value's size and SHA-256 digest, then exactly that many bytes. The receiver rejects missing or extra bytes and a wrong digest, and only then publishes the value atomically. Each worker moves at most four values at a time in each direction and accepts up to 64 transfer requests per direction; the rest of those wait their turn. A transfer that stops moving for 10 s fails; the overall limit is 120 s plus one second per 256 KiB, so large values on slow networks are not cut off while they are still moving.

IX. NETWORKING AND PROTOCOL

A. Messages

All control traffic uses 59 typed message classes, such as `TaskDispatch`, `TransferRequest` and `Heartbeat`. Each message is encoded as deterministic UTF-8 JSON, restricted to an allow-list of known classes, and sent as a frame with a 4-byte length prefix. Strict limits protect every receiver: frames of at most 1 MiB, identifiers of at most 256 bytes, at most 4 096 items per collection, a JSON depth of 32 and at most 65 536 JSON nodes. Messages carry a protocol version, and unknown versions are rejected.

B. Connections and Authentication

A node connects to the coordinator over TLS 1.3 [14], and both sides present certificates issued by the cluster's own certificate authority [15]. The node then proves that it holds its 256-bit secret by answering a fresh challenge with an HMAC-SHA256 code [16]. Challenges expire after 60 s and cannot be replayed. The coordinator listens only on its local network address and on loopback, never on every network interface, and the session warns when that address is reachable from the internet. Connections that submit, inspect or cancel programs are accepted only from the host itself.

Listing 3. Coordinator connection reader: wait, do not disconnect.

```
message = await self.stream.read_message()
# A full queue pauses reading: the peer is slowed by
# TCP flow control until this connection's messages
# are processed. Memory stays bounded either way.
await self.inbound.put(message)
```

Listing 4. Building the certificate pin of a join code (`enroll.py`).

```
def fingerprint(certificate_der: bytes) -> str:
    digest = hashlib.sha256(certificate_der).digest()[16:]
    return base64.urlsafe_b64encode(digest).decode(
        "ascii").rstrip("=")
```

C. Backpressure

Every connection has bounded queues of 64 messages. Early versions closed a connection when a queue was full. Stress testing showed that this dropped healthy workers: at the end of a large run, the burst of clean-up replies filled the queue. Now a full queue simply pauses reading, and TCP's own flow control slows the sender down (Listing 3). Workers likewise wait for room instead of failing, and skip a heartbeat when their queue is full, because the other messages already prove that they are alive. Memory stays bounded either way.

X. CLUSTER FORMATION AND MEMBERSHIP

A. Credentials

The first time a computer hosts, `dpr` uses `openssl` to create a private certificate authority on the NIST P-256 curve [17] and, from it, certificates for the coordinator, for the host's own operator connection, and for 32 worker identities, each valid for 825 days. Everything is created in a staging folder and moved into place only when complete. The authority's private key is then deleted, so nobody, not even the host, can create new identities later. Credentials are renewed automatically when fewer than 30 days remain.

B. Join Codes

When hosting starts, the host prints a join code of the form

```
dpr3_HOST:PORT:TOKEN:PIN
```

where `TOKEN` is 96 random bits, new every time hosting starts, and `PIN` is the first 128 bits of the SHA-256 digest of the host's certificate (Listing 4). The joining machine connects to the enrollment service, checks that the certificate it is shown matches the pin, and only then sends anything. A stranger on the network therefore cannot pose as the host.

C. Enrollment Service

The enrollment service is a small HTTPS service on its own port. It is hardened against misuse: at most eight requests are served at once, each request has a 20 s deadline and a 4 KiB size limit, a wrong token is answered only after one second, and at most eight machines may wait for approval at the same time.

D. Approval and Removal

A valid code alone admits no one (Fig. 5). The host's session shows `laptop-b` wants to join (482193) with

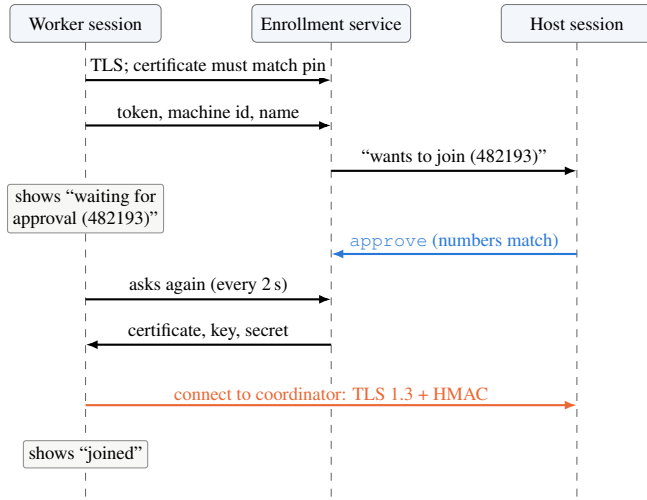


Fig. 5. Joining a cluster. The pin in the code protects against impostors; the matching pairing number lets the host see that the request is really from the machine in front of the other person.

TABLE IV
SESSION COMMANDS

Command	Effect
host	Start a cluster on this computer and show its join code
join <code>	Join a cluster (join alone rejoins)
run <file>	Run a program on the cluster (host only)
status	Show the cluster and its machines
approve <name>	Let a waiting machine join (host only)
kick <name>	Turn a waiting machine away or remove a member (host only)
exit	Stop dpr on this computer

a random six-digit pairing number, and the joining machine shows the same number. When the numbers match, the host types approve. The waiting machine asks again every two seconds for up to five minutes, and only the machine that asked can collect the approval. It then receives one of the 32 worker identities, connects to the coordinator and appears in status. A machine that joins again gets its previous identity back and can rejoin with just join. The kick command turns a waiting machine away or removes a member: it is disconnected at once, its identity is retired for good, including after restarts, and it is told so over its authenticated connection and forgets its credentials.

XI. USER INTERFACE

dpr is installed with `python install.py`, which falls back to a per-user install when the system refuses, and prints every command afterwards. Outside a session there are only two commands, `dpr start` and `dpr uninstall` (the latter removes the program and all its data after a confirmation). Everything else happens at the `dpr>` prompt (Table IV). Only one session can run at a time for each user account.

During a run the session shows the elapsed time, and `Ctrl-C` cancels the run on every computer. At the end it prints the program’s output and the run time, and saves a result file in `dpr-results/` next to the program, listing the output, the result and the tasks with the machine each ran on; the last 20

Listing 5. An illustrative host session in dpr’s output format (the name, address, code and time are made up; the code is shortened).

```

dpr> host
hosting 192.168.1.20:8740
code    dpr3_192.168.1.20:8750:q7Rk2...Tz9Qw
laptop-b wants to join (482193)
dpr> approve laptop-b
approved laptop-b
dpr> run primes/primes.py
primes: 78498
done in 10.0s, saved to primes/dpr-results/primes-...txt

```

TABLE V
THREATS AND PROTECTIONS

Threat	Protection
Eavesdropping	TLS 1.3 on every connection
Fake host or worker	Mutual certificates from a private CA; pinned join code; HMAC challenge
Replayed login	Fresh, expiring challenges
Guessing a join code	96-bit token; slow answers to wrong tokens
Unwanted member	Host approval with a matching pairing number
Removed member returning	Identity retired permanently
Remote control of runs	Program submission only from the host itself
Exposure to the internet	Listens on LAN address and loopback only; warning for public addresses
Hostile output text	Terminal control sequences removed
Private files leaving	Program folder only; home folder and drive roots refused; credential stores never packed
Secrets in process lists	Secrets passed through a pipe, never on the command line or in the environment; private files
Runaway task	Time and processor limits everywhere; memory and file-size limits on Linux and macOS (Section VIII)

results per program are kept. Output is shown for the first 1 024 tasks of a run, up to 8 KiB per task and stream. Text that comes from other machines (output, errors and machine names) is stripped of terminal control sequences before it is printed or saved, so it cannot rewrite the screen or disguise itself. Listing 5 shows a short illustrative session.

XII. SECURITY

A. Trust Model

dpr is built for a group of people who trust each other. The host runs its programs on every worker with the rights of the user who typed join (or as nobody when that user is root on Linux or macOS), and values pass between workers as Python objects. dpr is therefore defense in depth for a trusted group, not a sandbox for strangers’ code. Within that model it protects against eavesdroppers, impostors and mistakes (Table V).

B. Protecting the Local Machine

dpr keeps its data in a private folder (`~/ .dpr`). It refuses to adopt a folder that already contains other files, and it deletes only entries it created itself, so a mistyped path cannot destroy data. Nodes receive their secrets through a pipe from the session and exit with it.

C. Security Review

Version 5 of the project was a dedicated security and operating-system review of the whole code base, and later versions kept its rules. The security properties above are covered by tests, for example tests that reject a wrong secret, a certificate used under another name, a forged transfer event, and a removed member.

XIII. OPERATING-SYSTEM INTEGRATION

A central goal was that users never manage processes, ports or files by hand.

- *Supervision*: every background node is started with a pipe from its session and exits as soon as that pipe closes, whether the user types `exit`, closes the window or the session is killed.
- *Leftovers*: node process identifiers are recorded; the next `dpr start` stops anything an earlier, crashed session left behind.
- *Descendants*: processes started by tasks are cleaned up with process groups and parent-death signals on Linux, a parent-process watcher on macOS and a Job Object on Windows.
- *Ports*: the defaults are 8740 (control) and 8750 (joining) on the host and 8841 (data) on workers; a busy port is replaced by a free one.
- *Storage*: the package cache evicts old entries, worker data is discarded on restart, logs rotate at 1 MB, the history keeps 1 000 runs and the result folder 20 files per program.
- *Windows*: `dpr` works from Git Bash, finds the `openssl` that comes with Git for Windows, accepts paths such as `/c/Users/...`, and checks program paths against Windows rules on every platform.

XIV. IMPLEMENTATION

`dpr` is written for Python 3.12 and newer, using only the standard library: `asyncio` for networking [18], `ssl` and `hmac` for security, `ast` for analysis and `sqlite3` for history. It has no third-party runtime dependencies; only the host needs the `openssl` tool to create certificates. It is packaged with a standard `pyproject.toml` that installs a `dpr` command. A generated audit companion (`SOURCE-CODE.md`) contains the full text of every source file, and a test checks that it matches the release exactly. User documentation consists of a `README` and a short guide; each runtime component also has its own technical documentation. The project was developed in iterations, each one delivered as a complete, tested release (Section XVI).

XV. TESTING AND VERIFICATION

A. Test Levels

Testing was treated as part of the design, not an afterthought. The suite contains 1 841 automated tests that run with `pytest`; all of them pass on Python 3.12 and 3.13, in about 100 seconds each. Fig. 6 shows where they are. The tests work at five levels:

- *Unit tests* check pure logic, such as message validation, graph construction and scheduling rules.
- *Component tests* drive the coordinator with simulated worker connections and an injected clock, so timeouts can be tested

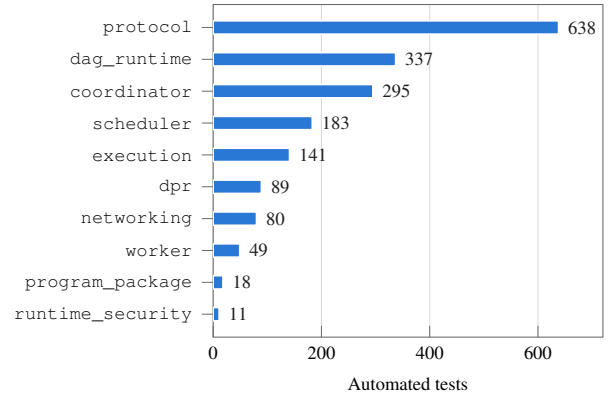


Fig. 6. The 1 841 automated tests by the package they are kept in. The protocol has the most tests because each of its 59 message types is tested separately: round trips, fragmented frames, immutability and malformed input.

Listing 6. An end-to-end regression test (shortened from `test_nodes.py`).

```
def test_wide_fan_in_large_values_and_back_to_back_runs(
    cluster_factory, tmp_path):
    cluster = cluster_factory(
        ["w1", "w2"], slots=2,
        node_args=("--inbound-queue", "4",
                   "--outbound-queue", "4"))
    first = asyncio.run(runs.run(cluster.config, program))
    assert first.output == "(160, 10485760) 3000000"
    second = asyncio.run(runs.run(cluster.config, simple))
    assert second.succeeded
    time.sleep(2) # let the end-of-run clean-up finish
    status = asyncio.run(cluster.status())
    assert sorted((w.worker_id, w.generation, w.online)
                  for w in status.workers) == [
        ("w1", 1, True), ("w2", 1, True)] # no reconnects
```

without waiting.

- *Transport tests* use real TLS sockets and freshly generated certificates.
- *Process tests* start real coordinator and worker processes on the local machine and run programs through them.
- *End-to-end tests* type into real `dpr start` sessions through pipes, exactly as a person would: hosting, joining, approving, running, kicking and exiting.

B. Regression Discipline

Each problem found in version 6 received a test that reproduces it. The test was first run against the old code to confirm that it fails there, and then against the fix in version 7. Listing 6 shows one of these tests: a real cluster with two workers must gather 160 values into one task, carry a 3 MB value between tasks, and run a second program right after the first without losing a worker. To make the test independent of machine speed, it shrinks the message queues to four entries, so every burst overflows them.

C. Stress Test

A separate stress program keeps every machine busy and checks every answer against a known value: it counts the primes below 100 million (5 761 455), finds the longest Collatz chain below 10 million (length 686, starting at 8 400 511), counts the points of a 1200×1200 Mandelbrot grid that stay bounded (241 831), computes four SHA-256 chains of 3 million rounds, creates four

TABLE VI
VERSIONS OF THE PROJECT

Version	Main change	Tests
v3 (1.0.0)	Complete runtime (analyzer, scheduler, coordinator, workers, data plane) with a command-line tool of 13 subcommands.	1 747
v4 (2.0.0)	New user interface: only <code>dpr start</code> and <code>dpr uninstall</code> outside a session; short, plain messages; automatic management of processes, caches and storage.	1 767
v5 (2.0.0)	Security and operating-system review of the whole code base, with fixes.	1 808
v6 (2.0.0)	Host approval with pairing numbers, and <code>kick</code> with permanent retirement of identities.	1 822
v7 (2.1.0)	Overload handling found by stress testing: transfer queueing and admission control, backpressure, larger values, size-aware deadlines, disk-sized storage, prompt clean-up.	1 841

data sets of about 9 MB that separate tasks check, and gathers 160 tiles of 200 KB into one task. The last line printed is `PASS` only if every check succeeds. It passed on the final version, both in the development environment and on the physical cluster of Section XVII.

XVI. DEVELOPMENT HISTORY

`dpr` grew in complete, tested versions (Table VI). Each version started from a concrete request and ended with the whole test suite passing; the suite of every version was run again for this report and passes on Python 3.12.

A. Problems Found by Stress Testing

The stress test of Section XV was written to break the system, and it did. Table VII lists the problems that it and the review that followed found in version 6, and how version 7 fixed them. The first problem was also noticed on the real cluster, where an in-place fix of the sender queue was applied and later merged into version 7.

XVII. PERFORMANCE EVALUATION

A. Setup

The measurements used three computers running Windows 11 on one local network. The host had 16 logical processor cores; because the host coordinates but does not run tasks, only the workers computed. Worker A had 4 logical cores of a 10th-generation Intel mobile processor, and worker B 4 logical cores of a 6th-generation Intel mobile processor, giving the cluster 8 slots in total. The *plain* baseline ran each program with `python` on the host, which uses one of its cores.

B. Method

A benchmark tool ran seven workloads (Table VIII) in both settings. For *plain*, it timed `python program.py` as a new process, including the interpreter’s start-up. For *dpr*, it submitted the same file to the cluster exactly as `run` does and timed from submission, including packing the program, until the result was back on the host. Each workload first ran once on the cluster as a *cold* run (when its code is shipped to the workers), reported separately. Then three timed repetitions followed; within each

TABLE VII
PROBLEMS FOUND IN VERSION 6 AND THEIR FIXES

Problem	Fix in version 7
A worker refused its fifth simultaneous outgoing transfer instead of waiting	Requests wait in a queue (up to 64); four are sent at a time
Gathering many results at once sent every transfer request together, so worker limits or transfer deadlines failed the run	Coordinator admits at most four transfers per worker and direction; the rest wait on the coordinator
The output of a self-contained task was limited to 1 MiB by a metadata limit	Outputs limited only by the value limit (256 MiB)
The reply burst at the end of a large run disconnected workers; an immediate second <code>run</code> found no machines	Full queues pause the sender instead of disconnecting
Cancelled or failed runs left some transfers on workers until they timed out	Clean-up is sent for every transfer at once
Fixed transfer deadlines could cut off large values on slow networks	Deadlines grow with the value size
A fixed 256 MiB result store per worker	A quarter of the free disk (256 MiB to 64 GiB)

TABLE VIII
BENCHMARK WORKLOADS

Workload	What it does
01 Latency	Two tiny statements; measures the fixed cost of a run
02 CPU, coarse	32 equal pure-Python loops of 10 million steps
03 CPU, fine	256 loops of 0.5 million steps (about 0.13 s each)
04 Data movement	8 blocks of 16 MiB made separately, then hashed by one task
05 Map-reduce	48 maps, each returning about 160 KB; one reduce
06 Dependency chain	12 steps, each needing the previous result
07 Mixed	The full stress test (214 tasks)

repetition every workload ran in both settings back to back, and the order of the two settings alternated between repetitions so that slow drift, such as heat, affected both equally. Medians are reported because they are robust to one disturbed run. Every run had to finish without error and print a checksum of its result, and all 49 runs did; the mixed workload also checks every answer against a known value (Section XV).

C. Results

Table IX and Fig. 7 give the median times; Fig. 8 shows the speedups. The raw data of all runs is listed in the appendix.

D. Discussion

1) *Parallel workloads*: The coarse CPU, map-reduce and mixed workloads ran 2.8 to 3.1 times faster, with small variation (coefficient of variation 2–5 % for the coarse and mixed workloads, 12 % for map-reduce). The mixed stress test dropped from 254 s to 83 s. Their parallel efficiency against the 8 slots was 0.35 to 0.38. Three factors keep it lower. First, a slot is not worth one host core. A slot is a logical core, and the processor models reported for both workers have two physical cores, each running two logical cores, so four busy slots do much less than four times the work of one. The workers also differ in speed:

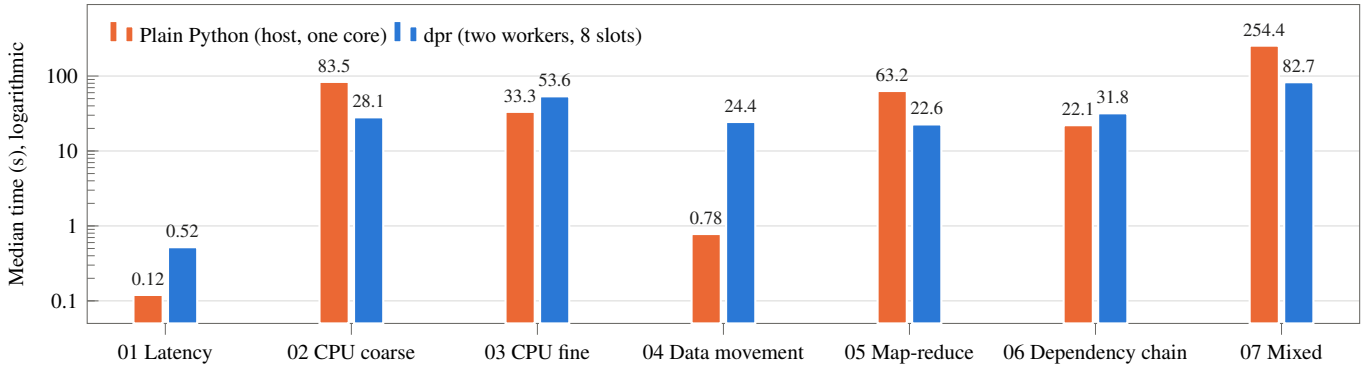


Fig. 7. Median wall-clock time of each workload, three repetitions each (lower is better; logarithmic scale). dpr is much faster on the three workloads with plenty of independent work, and slower where there is nothing to share, where tasks are tiny, or where data must cross the network.

TABLE IX
MEDIAN WALL-CLOCK TIMES (SECONDS) AND SPEEDUP

Workload	Plain	dpr	Cold	Speedup	E_8
01 Latency	0.12	0.52	0.51	0.23	—
02 CPU, coarse	83.53	28.12	26.65	2.97	0.37
03 CPU, fine	33.28	53.61	52.79	0.62	0.08
04 Data movement	0.78	24.37	25.81	0.03	—
05 Map-reduce	63.17	22.61	26.17	2.79	0.35
06 Dependency chain	22.08	31.82	20.38	0.69	—
07 Mixed	254.41	82.69	98.61	3.08	0.38

Speedup = plain median / dpr median. E_8 = speedup divided by the cluster's 8 slots, shown for workloads with parallel work.

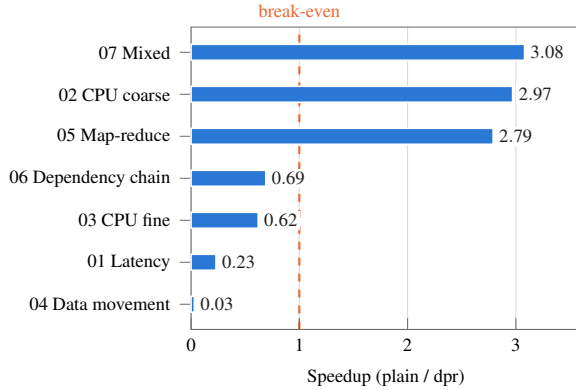


Fig. 8. Speedup of dpr over plain Python. Values above 1 mean the cluster was faster.

the dependency chain, which cannot run in parallel, took 22.1 s on the host, 31.8 s when 12 of its 14 tasks ran on worker B, and 20.4 s in the cold run, when 13 of its 14 tasks ran on worker A. Second, every task starts a new Python process. Third, the last tasks of a stage leave slots idle while they finish.

2) *Dependency chain*: With no parallel work, dpr cannot help, and the time depends mainly on which worker runs the chain. The cold run on worker A was even slightly faster than the host, which suggests that dpr's own overhead for twelve steps is small.

3) *Fine-grained tasks*: Each of the 256 tasks does only about 0.13 s of work, but each also needs a new Python process on a

worker, messages to and from the coordinator, and a transfer of its result to the final task. With 32 tasks per slot, this per-task overhead outweighs the gain. Tasks should take at least about a second each.

4) *Data movement*: In one process, making and hashing the eight 16 MiB blocks takes 0.8 s, and nothing has to move. In dpr the blocks are made on both workers, so the ones made on the other worker, about half of them, must cross the network before the final task can start. Even if all 128 MiB had crossed, the median time would correspond to only about 5.5 MB/s, and the times varied between 18.9 s and 28.8 s, which points to the network as the limit.

5) *Fixed cost*: A trivial program took 0.52 s, the fixed cost of packing, submitting, scheduling and collecting one run.

E. Earlier Measurements

Before the benchmark tool existed, two workloads were timed by hand with version 6 and the sender-queue fix: a prime-counting benchmark (109.4 s plain, 58.2 s with dpr, 1.88 $\times$) and a matrix benchmark (9.0 s plain, 6.6 s with dpr, 1.36 $\times$; all 589 824 result cells verified). The first run of the stress test on version 7 took 259.0 s plain and 82.3 s with dpr (3.15 $\times$).

F. Threats to Validity

The results come from one small cluster, three repetitions and one operating system. The efficiency compares laptop slots with one core of a different computer, so it describes this hardware as much as dpr. Single-machine baselines on the workers were not collected, and the network speed was not measured separately. The absolute numbers should therefore be read as an example; the pattern (strong gains for coarse independent work, losses for tiny tasks, sequential chains and data-heavy work) is the general result.

XVIII. LIMITATIONS

- Only the file that is run is analyzed; functions imported from other files are treated as unknown code.
- A top-level `import`, `print` or other effect ends the parallel part of a program; imports have to be placed inside `@task` functions.
- The host does not run tasks, so its cores stay idle.

- Each self-contained task starts a new Python process, so tasks much shorter than a second are slower on the cluster than in one process.
- Output is shown only for the first 1 024 tasks of a run, and only up to 8 KiB per task and stream.
- Libraries a program imports must be installed on every worker, and the same versions are not enforced.
- If the host stops, the running program is lost; there is no recovery of an active run after a coordinator crash.
- dpr is designed for trusted members, not for running strangers' code.

XIX. FUTURE WORK

- Let the host also run tasks, adding its cores to the cluster.
- Recognize well-known standard-library imports as harmless, so programs with imports at the top can still run in parallel.
- Reuse worker processes between tasks to cut the start-up cost of small tasks.
- Check before a run that every worker has the program's libraries, and install them automatically from a `requirements.txt`.
- Analyze local modules imported by the program, not only the main file.
- Measure network speed and use it when placing data-heavy tasks.

XX. CONCLUSION

dpr shows that ordinary Python programs can be run in parallel across a few everyday computers without rewriting them, as long as the system is honest about what it can prove. By analyzing the program conservatively, running only proven or declared self-contained work elsewhere, and keeping everything else in its original order, dpr gives the same result as a normal run, provided the `@task` promises are true and the program does not depend on the computer it runs on. Around that core, the project built a complete and careful system: secure cluster formation with approval and removal, a coordinator that survives slow and disappearing machines, a direct and verified data channel, bounded resources with backpressure, a two-command interface, and more than 1 800 tests. On two modest laptops it made compute-heavy programs about three times faster. Its limits are clear and measured, and they point directly to the next steps.

APPENDIX A: ALL BENCHMARK RUNS

Table X lists every measured run in seconds. Rep. 1 and 3 ran plain first; rep. 2 ran dpr first.

TABLE X
ALL MEASURED RUNS (SECONDS)

Workload	Plain			dpr			
	1	2	3	Cold	1	2	3
01	0.13	0.12	0.10	0.51	0.50	0.52	0.68
02	85.41	83.53	83.05	26.65	26.51	28.12	28.97
03	33.72	33.28	32.65	52.79	54.68	53.10	53.61
04	0.87	0.78	0.77	25.81	18.88	24.37	28.80
05	62.36	63.17	64.06	26.17	22.61	27.59	22.26
06	22.08	21.29	22.25	20.38	31.49	31.82	32.13
07	254.41	248.85	257.54	98.61	82.74	82.69	79.27

APPENDIX B: DEFAULT LIMITS AND TIMEOUTS

TABLE XI
MAIN DEFAULTS

Setting	Default
Heartbeat interval / worker lost after	1 s / 30 s
Task attempts	2
Task time / memory (Linux, macOS)	1 day / ≥ 2 GiB
Output shown per task and stream	8 KiB
Value size	256 MiB
Result store per worker	$\frac{1}{4}$ free disk (256 MiB–64 GiB)
Program folder	16 MiB, 8 MiB/file, 2 048 files
Package cache	256 MiB
Transfers per worker and direction	4
Transfer requests per worker and direction	64
Transfer stall limit / overall limit	10 s / 120 s + 1 s per 256 KiB
Control message / queue	1 MiB / 64 messages
Worker identities per host	32
Machines waiting for approval	8
Approval wait	5 minutes
Credential validity / renewal	825 days / 30 days before expiry
Run history / result files	1 000 runs / 20 per program

REFERENCES

- [1] Python Software Foundation, “Glossary: global interpreter lock,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/glossary.html>
- [2] Python Software Foundation, “multiprocessing—Process-based parallelism,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/library/multiprocessing.html>
- [3] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008.
- [4] M. Zaharia *et al.*, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2012, pp. 15–28.
- [5] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proc. 14th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2015, pp. 130–136.
- [6] P. Moritz *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. 13th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, USA, 2018, pp. 561–577.
- [7] A. B. Kahn, “Topological sorting of large networks,” *Commun. ACM*, vol. 5, no. 11, pp. 558–562, Nov. 1962.
- [8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, USA, 1967, pp. 483–485.

- [9] Python Software Foundation, “ast—Abstract syntax trees,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/library/ast.html>
- [10] Python Software Foundation, “The import statement,” *The Python Language Reference, Python 3.12*. [Online]. Available: https://docs.python.org/3.12/reference/simple_stmts.html
- [11] Python Software Foundation, “sqlite3—DB-API 2.0 interface for SQLite databases,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/library/sqlite3.html>
- [12] Python Software Foundation, “pickle—Python object serialization,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/library/pickle.html>
- [13] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.
- [14] E. Rescorla, “The Transport Layer Security (TLS) protocol version 1.3,” IETF, RFC 8446, Aug. 2018.
- [15] D. Cooper *et al.*, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” IETF, RFC 5280, May 2008.
- [16] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” IETF, RFC 2104, Feb. 1997.
- [17] National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS PUB 186-4, Jul. 2013.
- [18] Python Software Foundation, “asyncio—Asynchronous I/O,” *Python 3.12 Documentation*. [Online]. Available: <https://docs.python.org/3.12/library/asyncio.html>